



南开大学
Nankai University

南 开 大 学

计算机网络实验报告

实验 1：利用流式套接字编写聊天程序

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：徐敬东、张建忠

2025 年 11 月 19 日

目录

1 实验要求	1
2 通信协议设计	1
2.1 协议要点	1
2.2 报文格式（按行文本）	1
2.3 流程与异常	2
3 系统设计与实现	2
3.1 整体架构	2
3.1.1 组件与职责	2
3.1.2 线程模型	2
3.2 服务器实现	2
3.3 客户端实现	5
4 实验过程与结果	6
4.1 实验环境与准备	6
4.2 编译与运行	6
4.3 用例与结果总览	7
4.4 关键用例详述	7
4.5 抓包与丢包观察	11
5 实验总结与心得体会	12

1 实验要求

1. 设计聊天协议，并给出聊天协议的完整说明。
2. 利用 C 或 C++ 语言，使用基本的 Socket 函数进行程序编写，不允许使用 CSocket 等封装后的类。
3. 程序应有基本的对话界面，但可以不是图形界面。程序应有正常的退出方式。
4. 完成的程序应能支持英文和中文聊天。
5. 采用多线程，支持多人聊天。
6. 编写的程序应结构清晰，具有较好的可读性。
7. 在实验中观察是否有数据包的丢失，提交程序源码、可执行代码和实验报告。

2 通信协议设计

2.1 协议要点

- 协议：SCCP v1.0；传输层：TCP（端口 9090）；编码：UTF-8/控制台。
- 客户端**首包发送昵称**（行文本，结尾 `\n`）；后续**按行文本**发送聊天。
- 服务器对所有下发消息**在尾部追加时间戳**并广播；客户端本地以“[我] …”回显（不经网络）。
- 空消息丢弃；消息体建议 $\leq 1023\text{B}$ （实现采用“截断”策略）。

2.2 报文格式（按行文本）

边界规则：以单个 `\n` 作为消息分隔。客户端发送前**主动追加** `\n`；服务器接收后**先移除末尾** `\r/\n`，再格式化并**补回一个** `\n` 广播。

字段定义

字段	类型	取值/长度	说明
消息内容	字符串	$\leq 1023\text{B}$	客户端原始输入（ <code>getline</code> 已去除行尾）
结束符	1B	<code>\n</code> (0x0A)	客户端主动追加的分隔符

示例

- C→S（聊天）：<内容>`\n`
- S→C（用户消息）：[<用户名>] <内容> [YYYY-MM-DD HH:MM:SS]`\n`
- S→C（系统消息）：[系统] <提示> [YYYY-MM-DD HH:MM:SS]`\n`
- 本地回显（不走网络）：[我] <内容> [YYYY-MM-DD HH:MM:SS]`\n`

2.3 流程与异常

交互总览

阶段	客户端动作	服务器动作
上线	1. 建立 TCP 连接 2. 首包发送 < 用户名 > \n	1. 记录昵称并绑定连接 2. 回本人: [系统] < 用户名 > 已成功建立连接。 [YYYY-MM-DD HH:MM:SS] \n 3. 广播他人: [系统] < 用户名 > 加入了聊天室。 [YYYY-MM-DD HH:MM:SS] \n
聊天	1. 发送 < 内容 > \n	1. 去尾 \r/\n 2. 组装 [< 用户名 >] < 内容 > [YYYY-MM-DD HH:MM:SS] \n 3. 广播 (排除发送者)
退出	1. 输入 quit, 本地 closesocket 2. 接收线程下一次 $recv \leq 0$ 时提示 “[系统] 与服务器的连接已断开。”	1. 会话线程 $recv \leq 0$, 从在线表移除 2. 广播: [系统] < 用户名 > 离开了聊天室。 [YYYY-MM-DD HH:MM:SS] \n 3. 关闭连接

并发与同步:服务器一连一线程;客户端主线程发送 + 接收线程打印。在线表与广播使用 `std::mutex` 保护 (可在实现层先复制连接列表以缩短持锁时间)。

3 系统设计与实现

3.1 整体架构

本程序采用**客户端-服务器**模型, 消息以“**单行文本 + \n**”作为边界; 服务器负责集中转发与在线会话管理, 客户端负责输入采集与输出显示。

3.1.1 组件与职责

- **服务器:** 监听端口 9090; 每个连接创建会话线程; 维护在线表 (套接字 + 昵称); 格式化并广播消息; 在连接建立/断开时发送系统消息。
- **客户端:** 连接服务器并首包发送昵称; 创建接收线程阻塞读取服务器消息; 主线程读取用户输入并发送; 本地仅回显 “[我] …” 一行。

3.1.2 线程模型

服务器“**主线程监听 + 每连接一线程**”; 客户端“**主线程发送 + 接收线程打印**”。

3.2 服务器实现

服务器：初始化与监听（主线程骨架）

```
1 int main() {
2     // 1) 初始化 Winsock
3     WSADATA wsa;
4     WSASStartup(MAKEWORD(2,2), &wsa);
5
6     // 2) 创建监听套接字 (IPv4/TCP)
7     SOCKET listen_fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
8
9     // 3) 允许端口快速复用, 避免 TimeWait/占用
10    BOOL reuse = TRUE;
11    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, (char*)&reuse, sizeof(reuse));
12
13    // 4) 绑定到 0.0.0.0:9090
14    sockaddr_in addr{};
15    addr.sin_family = AF_INET;
16    addr.sin_port = htons(9090);
17    addr.sin_addr.s_addr = INADDR_ANY;
18    bind(listen_fd, (sockaddr*)&addr, sizeof(addr));
19
20    // 5) 开始监听
21    listen(listen_fd, SOMAXCONN);
22    puts("服务器已启动, 端口是 9090, 等待客户端的连接……");
23
24    // 6) 循环接受连接: 每个连接独立线程处理
25    for (;;) {
26        SOCKET cli = accept(listen_fd, nullptr, nullptr);
27        std::thread(client_thread, cli).detach(); // 一连一线程
28    }
29 }
```

服务器：在线用户与广播

```
1 struct Client { SOCKET s; std::string name; };
2 std::vector<Client> g_clients;
3 std::mutex g_mtx;
4
5 // 广播消息: except 表示不需要发送的套接字 (通常是消息的发送者)
6 void broadcast(const std::string& msg, SOCKET except = INVALID_SOCKET) {
7     // 复制句柄列表以缩短持锁时间
8     std::vector<SOCKET> sockets;
9     {
10         std::lock_guard<std::mutex> lk(g_mtx);
11         for (auto &c : g_clients)
12             if (c.s != except) sockets.push_back(c.s);
13     }
14     // 非临界区逐个发送
15     for (SOCKET s : sockets) {
16         send(s, msg.c_str(), (int)msg.size(), 0);
17     }
18 }
```

通用工具：时间戳与按行接收

```
1 // 统一格式的本地时间戳, 供 [系统]/[用户名]/[我] 消息尾部使用
```

```

2 std::string now_str() {
3     std::time_t t = std::time(nullptr);
4     std::tm lt{}; localtime_s(&lt, &t);
5     char buf[32]; std::strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", &lt);
6     return buf;
7 }
8
9 // 从 TCP 流中读取一条“以 '\n' 结束”的消息，自动丢弃 '\r'
10 std::string recv_line(SOCKET s) {
11     std::string out; char ch;
12     for (;;) {
13         int n = recv(s, &ch, 1, 0); // 阻塞读 1 字节，稳妥处理边界
14         if (n <= 0) return std::string(); // <=0 表示断开或错误
15         if (ch == '\n') break; // 行结束
16         if (ch != '\r') out.push_back(ch);
17     }
18     return out;
19 }

```

服务器：会话线程 client_thread

```

1 void client_thread(SOCKET cli) {
2     // 1) 首包昵称 (空置 guest)
3     std::string name = recv_line(cli);
4     if (name.empty()) name = "guest";
5
6     // 加入在线表 (受互斥保护)
7     {
8         std::lock_guard<std::mutex> lk(g_mtx);
9         g_clients.push_back({cli, name});
10    }
11
12    // 2) 确认本人 & 广播加入 (统一在尾部追加时间戳与 '\n')
13    std::string t = now_str();
14    std::string self_ok = "[系统] " + name + " 已成功建立连接。[" + t + "]\n";
15    send(cli, self_ok.c_str(), (int)self_ok.size(), 0);
16
17    std::string join_msg = "[系统] " + name + " 加入了聊天室。[" + t + "]\n";
18    broadcast(join_msg, cli); // except=cli, 排除发送者
19
20    // 3) 聊天循环: 逐行接收 -> 格式化 -> 广播 (排除发送者)
21    for (;;) {
22        std::string text = recv_line(cli);
23        if (text.empty()) break; // <=0: 对端断开或错误, 跳出循环
24        std::string line = "[" + name + " ] " + text + " [" + now_str() + "]\n";
25        broadcast(line, cli);
26    }
27
28    // 4) 清理与“离开”广播
29    {
30        std::lock_guard<std::mutex> lk(g_mtx);
31        g_clients.erase(std::remove_if(g_clients.begin(), g_clients.end(),
32            [&](const Client& c){ return c.s == cli; }), g_clients.end());
33    }
34    std::string left = "[系统] " + name + " 离开了聊天室。[" + now_str() + "]\n";

```

```

35 broadcast(left, cli);
36 closesocket(cli);
37 }

```

3.3 客户端实现

客户端：连接与注册（首包昵称）

```

1 SOCKET connect_server(const char* ip="127.0.0.1", uint16_t port=9090) {
2     WSADATA wsa; WSASStartup(MAKEWORD(2,2), &wsa);
3     SOCKET s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
4     sockaddr_in svr{}; svr.sin_family = AF_INET;
5     svr.sin_port = htons(port); inet_pton(AF_INET, ip, &svr.sin_addr);
6     connect(s, (sockaddr*)&svr, sizeof(svr));
7     return s;
8 }
9
10 void send_name(SOCKET s, const std::string& name) {
11     // 首包即昵称（行文本协议：追加 '\n' 作为边界）
12     std::string n = name.empty() ? "guest" : name;
13     n.push_back('\n');
14     send(s, n.c_str(), (int)n.size(), 0);
15 }

```

客户端：接收线程 recv_thread

```

1 std::atomic<bool> g_running{true};
2
3 // 接收线程：原样打印服务器消息；当对端关闭/网络异常时，提示并退出
4 void recv_thread(SOCKET s) {
5     char buf[1024]; int n;
6     while ((n = recv(s, buf, sizeof(buf)-1, 0)) > 0) {
7         buf[n] = 0;
8         fputs(buf, stdout); // 服务器侧已追加时间戳与 '\n'
9     }
10    puts("[系统] 与服务器的连接已断开。");
11    g_running = false;
12 }

```

客户端：发送路径与“仅留一行回显”

```

1 // 清除上一行：用 WinAPI 将上一行填空并把光标移到行首，避免“原始输入 + 回显”双行
2 void clear_prev_line() {
3     HANDLE h = GetStdHandle(STD_OUTPUT_HANDLE);
4     CONSOLE_SCREEN_BUFFER_INFO csbi;
5     if (!GetConsoleScreenBufferInfo(h, &csbi)) return;
6     COORD pos = csbi.dwCursorPosition;
7     if (pos.Y > 0) pos.Y -= 1, pos.X = 0; // 上一行行首
8     DWORD written = 0;
9     FillConsoleOutputCharacter(h, ' ', csbi.dwSize.X, pos, &written);
10    SetConsoleCursorPosition(h, pos);
11 }
12
13 // 主线程：读一行 -> 本地回显 -> 发送到服务器

```

```
14 void send_loop(SOCKET s, const std::string& myname) {
15     std::thread(recv_thread, s).detach(); // 启动接收线程
16     puts("您已成功进入聊天室，现在开始聊天吧~");
17     puts("请输入消息，输入 quit 退出。");
18
19     std::string line;
20     while (g_running && std::getline(std::cin, line)) {
21         if (line == "quit") break; // 本地退出，不再发送
22         clear_prev_line(); // 擦掉原输入，只留一行回显
23         printf("[我] %s [%s]\n", line.c_str(), now_str().c_str());
24         line.push_back('\n'); // 行分隔：追加 '\n' 再发送
25         send(s, line.c_str(), (int)line.size(), 0);
26     }
27     closesocket(s); // 主动关闭，接收线程将感知 recv<=0
28 }
```

4 实验过程与结果

本章节基于当前实现（端口 9090、首包昵称、单行文本 + \n 为消息边界、服务器统一在尾部追加时间戳、客户端本地仅回显 “[我] …” 一行）进行验证。我们先给出环境与编译方式，再按测试用例（步骤 → 预期 → 实际）展示结果，最后给出抓包与丢包观察。

4.1 实验环境与准备

- **操作系统/工具：**Windows 10/11；Visual Studio 2022（命令行编译 cl），Winsock2；控制台中文环境。
- **网络拓扑：**本机回环（127.0.0.1）或同一局域网多台主机。
- **源代码：**chatroom_server.cpp、chatroom_client.cpp；协议文档《SCCP v1.0》。

4.2 编译与运行

```
1 REM 在“x64 Native Tools Command Prompt”或 VS 开发者命令行中执行
2 cl /EHsc chatroom_server.cpp /link ws2_32.lib
3 cl /EHsc chatroom_client.cpp /link ws2_32.lib
4
5 REM 运行顺序：先启动服务器，再启动一个或多个客户端
6 chatroom_server.exe
7 chatroom_client.exe
```


4.3 用例与结果总览

编号	用例	关键预期	结论
TC-01	服务器启动	控制台打印“服务器已启动, 端口是 9090, 等待客户端的连接……”	通过
TC-02	客户端连接与昵称注册	首包昵称; 本人收到 “[系统] ...已成功建立连接。[时]”	通过
TC-03	加入广播	其他在线端收到 “[系统] <name> 加入了聊天室。[时]”	通过
TC-04	中文消息转发	发送端本地仅 “[我] 内容 [时]”; 他人端 “[name] 内容 [时]”	通过
TC-05	英文/混合内容	多语言显示正常, 无乱码	通过
TC-06	时间戳一致性	所有可见消息行尾均为 YYYY-MM-DD HH:MM:SS	通过
TC-07	正常退出 (quit)	他人端 “[系统] <name> 离开了聊天室。[时]”; 本端提示断开	通过
TC-08	服务器异常退出	客户端提示 “[系统] 与服务器的连接已断开。”	通过
TC-09	并发 (≥ 3 客户端)	同时聊天无阻塞, 无串行卡顿	通过
TC-10	边界与长度	空行丢弃; 超长消息按“截断”策略处理	通过

4.4 关键用例详述

TC-01 服务器启动

步骤: 运行 chatroom_server.exe。

预期: 控制台打印“服务器已启动, 端口是 9090, 等待客户端的连接……”。

实际: 与预期一致。

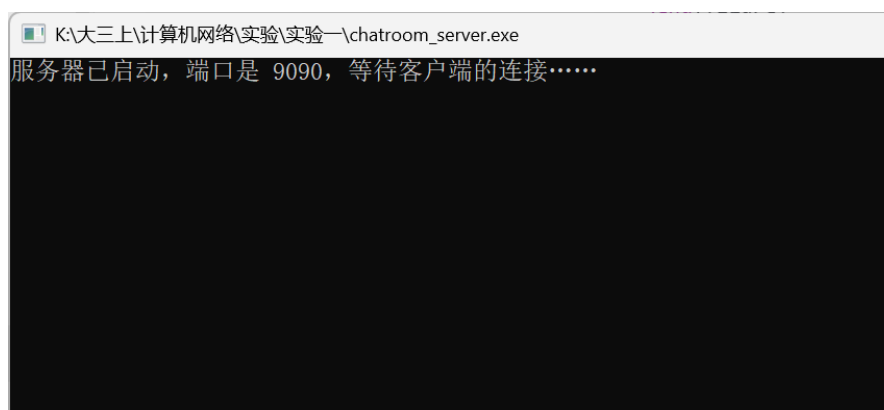


图 1: 服务器启动

TC-02 客户端连接与昵称注册

步骤：运行 chatroom_client.exe，输入昵称（如“张三”），回车。

预期：本端接收“[系统] 张三已成功建立连接。[YYYY-MM-DD HH:MM:SS]”。

实际：与预期一致；若输入为空则被置为 guest。

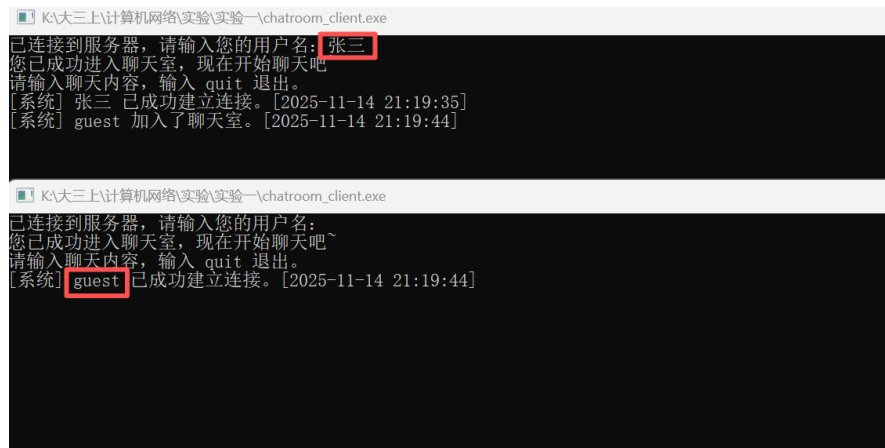


图 2: 客户端连接与昵称注册

TC-03 加入广播

步骤：在第二个客户端输入昵称“李四”进入。

预期：除“李四”本人外，其他在线端显示“[系统] 李四加入了聊天室。[时间]”。

实际：与预期一致。

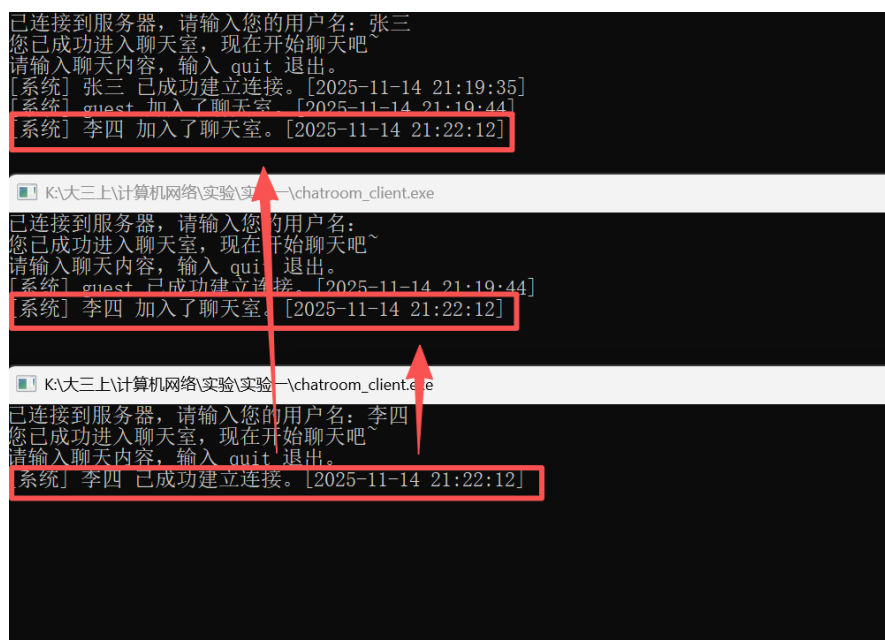


图 3: 加入广播

TC-04 中文消息转发与本地回显

步骤：张三端输入“你好”。

预期：张三端只出现“[我] 你好 [时间]”（上一行原始输入被擦除）；他人端显示“[张三] 你好 [时间]”。

实际：与预期一致；本地仅一行回显。

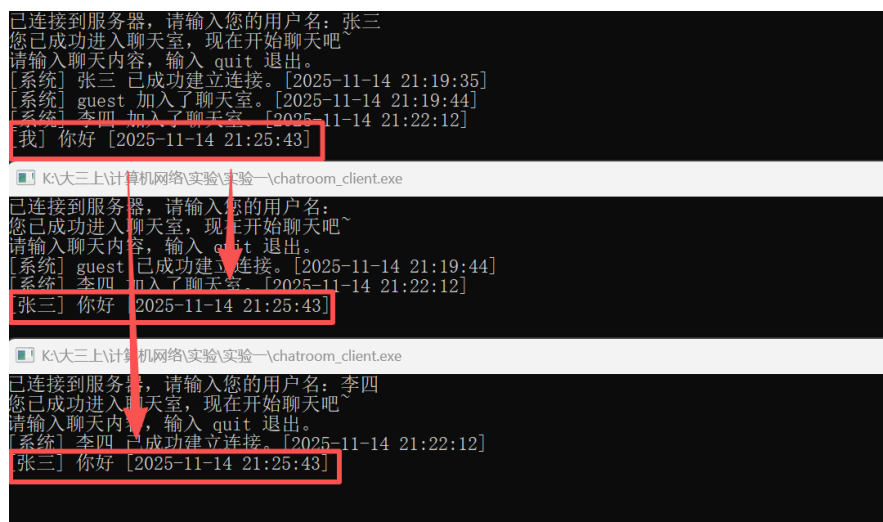


图 4: 中文消息转发与本地回显

TC-05 英文/混合内容

步骤：李四端输入“hello 早上好”。

预期：多语言混排正常；两端均无乱码。

实际：与预期一致。

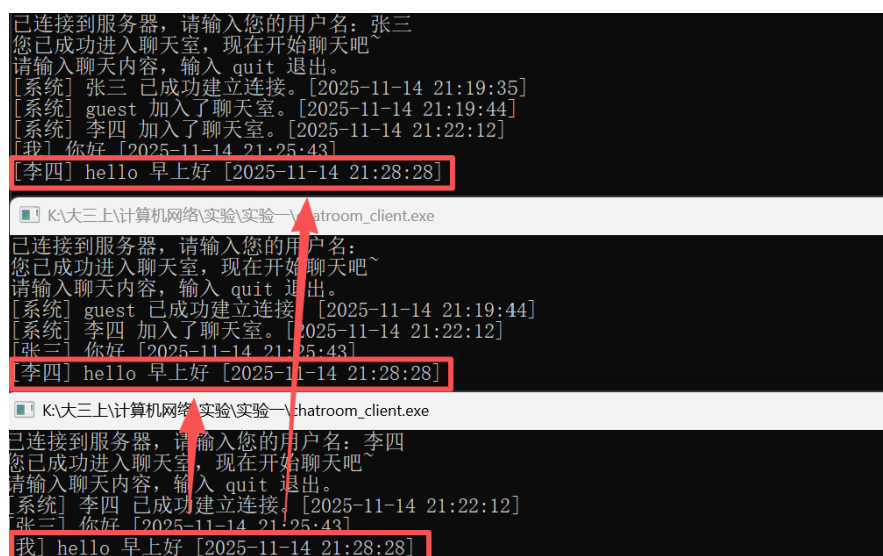


图 5: 混合内容输入

TC-06 时间戳一致性

步骤：任意客户端发送 2-3 条消息；观察各端显示。

预期：所有消息（系统 + 用户 + 本地回显）行尾均为 YYYY-MM-DD HH:MM:SS。

实际：一致。

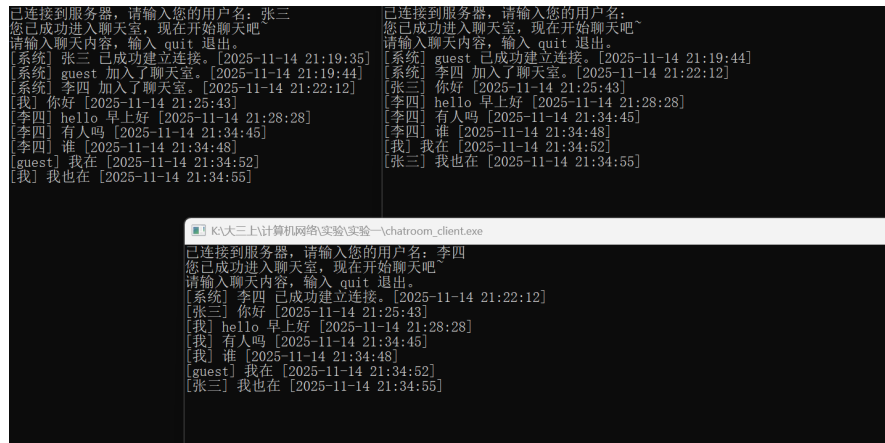


图 6: 时间戳一致性

TC-07 正常退出 (quit)

步骤：任一客户端输入 quit。

预期：本端：提示“[系统] 与服务器的连接已断开。”；他人端：收到“[系统] <name> 离开了聊天室。[时间]”。

实际：与预期一致。

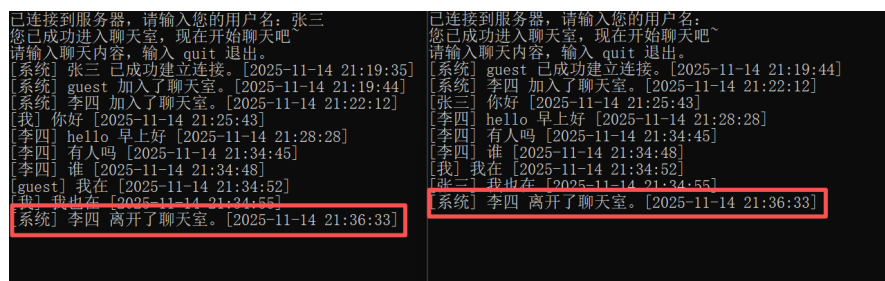


图 7: 正常退出

TC-08 服务器异常退出

步骤：直接关闭服务器控制台窗口或 Ctrl+C 终止。

预期：客户端 recv 返回 ≤ 0 并提示断开。

实际：与预期一致。

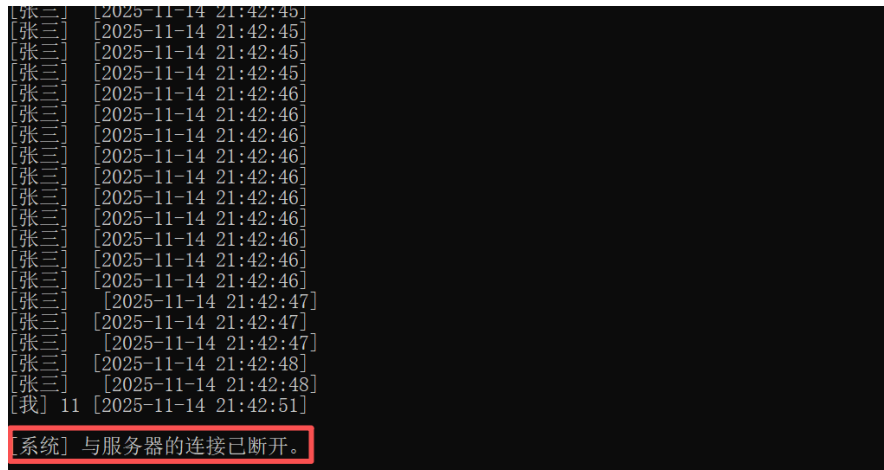


图 8: 服务器异常退出

TC-09 边界与长度策略

步骤：发送空行与超长文本（1023B）。

预期：空行丢弃；超长按“截断”策略（显示前 1023B）。

实际：与预期一致。



图 9: 边界与长度策略

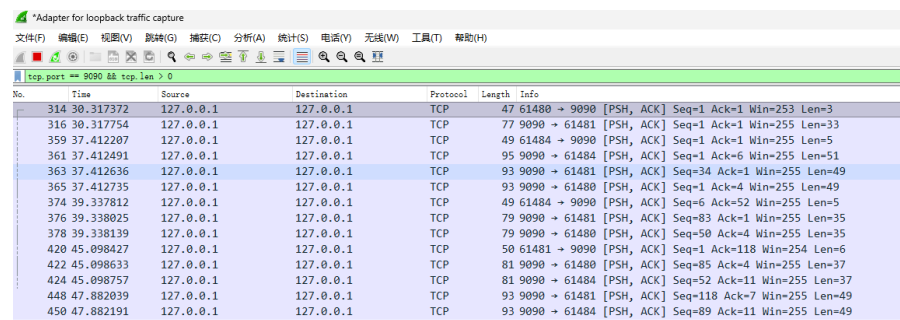
4.5 抓包与丢包观察

工具：Wireshark。

过滤表达式: tcp.port == 9090

操作与观察：

1. 启动抓包后依次执行 TC-02~TC-09；检查 TCP 流，无异常重传 & Reset。
2. **结果：**在本机与局域网环境下，未观察到丢包；TCP 会话正常三次握手/四次挥手，消息序列与时间戳一致。



No.	Time	Source	Destination	Protocol	Length	Info
314	30.317372	127.0.0.1	127.0.0.1	TCP	47	61480 → 9090 [PSH, ACK] Seq=1 Ack=1 Win=253 Len=3
316	30.317754	127.0.0.1	127.0.0.1	TCP	77	9090 → 61481 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=33
359	37.412207	127.0.0.1	127.0.0.1	TCP	49	61484 → 9090 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=5
361	37.412491	127.0.0.1	127.0.0.1	TCP	95	9090 → 61484 [PSH, ACK] Seq=1 Ack=6 Win=255 Len=51
363	37.412636	127.0.0.1	127.0.0.1	TCP	93	9090 → 61481 [PSH, ACK] Seq=34 Ack=1 Win=255 Len=49
365	37.412735	127.0.0.1	127.0.0.1	TCP	93	9090 → 61480 [PSH, ACK] Seq=1 Ack=4 Win=255 Len=49
374	39.337812	127.0.0.1	127.0.0.1	TCP	49	61484 → 9090 [PSH, ACK] Seq=6 Ack=52 Win=255 Len=5
376	39.338025	127.0.0.1	127.0.0.1	TCP	79	9090 → 61481 [PSH, ACK] Seq=83 Ack=1 Win=255 Len=35
378	39.338139	127.0.0.1	127.0.0.1	TCP	79	9090 → 61480 [PSH, ACK] Seq=50 Ack=4 Win=255 Len=35
420	45.098427	127.0.0.1	127.0.0.1	TCP	50	61481 → 9090 [PSH, ACK] Seq=1 Ack=118 Win=254 Len=6
422	45.098633	127.0.0.1	127.0.0.1	TCP	81	9090 → 61480 [PSH, ACK] Seq=85 Ack=4 Win=255 Len=37
424	45.098757	127.0.0.1	127.0.0.1	TCP	81	9090 → 61484 [PSH, ACK] Seq=52 Ack=11 Win=255 Len=37
448	47.882039	127.0.0.1	127.0.0.1	TCP	93	9090 → 61481 [PSH, ACK] Seq=118 Ack=7 Win=255 Len=49
450	47.882191	127.0.0.1	127.0.0.1	TCP	93	9090 → 61484 [PSH, ACK] Seq=89 Ack=11 Win=255 Len=49

图 10: 抓包与丢包

5 实验总结与心得体会

通过本次“基于流式套接字的聊天程序”实验，我对计算机网络中套接字编程和应用层协议有了更直观的认识。实验中需要自行约定消息格式和处理流程，我真正体会到：TCP 只提供字节流，消息边界和语义都要由应用层自己定义，如果设计不清晰，很容易在收发两端产生歧义和 bug。

在实现过程中,我完成了客户端与服务器的连接建立、消息发送与接收等功能,理解了 `socket`、`bind`、`listen`、`accept`、`connect` 等调用背后的逻辑。为了支持多用户同时聊天，我采用了多线程模型处理并发连接，并通过适当的同步手段保证共享数据的正确性，这让我对并发编程有了初步实践体验。

通过测试和调试，我也意识到细节的重要性，比如异常断开时的资源释放、输入输出的边界处理等，都直接影响程序的健壮性。整体来看，本次实验帮助我把课堂上抽象的网络协议和系统调用“落地”成了一个真正可运行的程序，增强了动手能力，也激发了我进一步学习网络编程的兴趣。